

Gestor de Turnos — Club Once Unidos

Sistema de reservas online para canchas de tenis

Cliente: Club Once Unidos, Mar del Plata

Rol: Desarrollo completo (diseño, frontend, backend, deploy)

Tiempo de desarrollo: ~20 horas (10 días, 1.5–2 hs/día, como side project)

Estado: En producción, fase piloto

Demo: gestorturnos.vercel.app

Código: github.com/nicopetito/gestor-turnos

El problema

El Club Once Unidos gestiona 3 canchas de tenis con un horario de 8:00 a 22:00 hs — hasta 42 turnos disponibles por día, más de 1.200 al mes en capacidad máxima.

Antes del sistema, todo se coordinaba por **WhatsApp y una planilla de Excel**: los socios mandaban mensajes para reservar, el encargado revisaba disponibilidad manualmente, confirmaba, y actualizaba la planilla. Esto generaba varios problemas:

- **Reservas duplicadas:** sin una fuente de verdad centralizada, dos personas podían pedir el mismo turno al mismo tiempo.
 - **Tiempo del encargado:** responder mensajes, cruzar horarios y actualizar la planilla consumía atención constante durante las 14 horas de operación diaria.
 - **Falta de visibilidad:** los socios no tenían forma de ver qué turnos estaban libres antes de escribir — cada consulta dependía de una respuesta humana.
 - **Errores y olvidos:** sin confirmación automática, las reservas se perdían o se confundían.
-

La solución

Un sistema web donde los socios reservan directamente, ven la disponibilidad en tiempo real, y reciben confirmación automática por email — sin intermediarios.

Funcionalidades principales

- **Vista de disponibilidad en tiempo real:** calendario visual donde el usuario ve al instante qué turnos están libres, ocupados o bloqueados.

- **Reserva self-service 24/7:** cualquier socio puede reservar sin depender del horario ni la disponibilidad del encargado.
 - **Panel de administración:** el administrador del club tiene un dashboard completo donde puede gestionar canchas, horarios, bloquear turnos, ver estadísticas de uso y administrar reservas.
 - **Confirmación automática por email:** vía Resend, cada reserva dispara un email de confirmación al usuario.
 - **Sincronización automática con Excel:** el sistema actualiza la planilla de Excel que el club ya usaba. Esto fue una decisión deliberada: no obligar al club a abandonar su herramienta conocida de un día para el otro, sino darles una transición gradual con respaldo.
 - **Diseño responsive con enfoque diferenciado:** la versión mobile y desktop no son la misma UI achicada — cada una está pensada para su contexto de uso, priorizando funcionalidad sobre uniformidad visual.
-

Decisiones técnicas

Stack

Tecnología	RoI
Next.js 16 (App Router)	Framework principal, SSR + API Routes
TypeScript	Tipado estricto en todo el proyecto
Supabase (Postgres)	Base de datos, autenticación, Row Level Security
Tailwind CSS	Estilos utilitarios, responsive design
Resend	Emails transaccionales
date-fns v4	Manejo de fechas y horarios
Vercel	Deploy y hosting serverless

¿Por qué Supabase y no Firebase?

Necesitaba una base de datos relacional de verdad. El modelo de turnos tiene relaciones claras entre canchas, horarios, usuarios y reservas — en Firestore (NoSQL) hubiera tenido que desnormalizar y duplicar datos. Supabase me da Postgres completo con JOINS, Row Level

Security en SQL, y un tier gratuito que cubre de sobra las necesidades del club. Además, al ser open source, el club no queda atado a un proveedor.

¿Por qué sincronizar con la planilla de Excel?

Decisión de producto, no técnica. El club ya tenía un flujo armado alrededor de esa planilla. Forzarlos a abandonarla el día uno iba a generar fricción y resistencia. La sincronización automática les permite hacer la transición a su ritmo: el sistema nuevo es la fuente de verdad, pero la planilla sigue ahí como respaldo y como interfaz familiar para quien la prefiera.

¿Por qué no requiere login para los usuarios?

Reducir fricción al máximo. Para un club con una base chica de socios, agregar registro + login + recuperación de contraseña era más barrera que valor. El admin sí tiene acceso autenticado al panel de gestión.

Métricas técnicas

Resultados de Lighthouse (desktop):

Métrica	Score
Performance	99
Accesibilidad	95
Buenas prácticas	100
SEO	100

Arquitectura serverless en Vercel: escala automáticamente sin configuración de infraestructura. Costo operativo actual: \$0.

Contexto personal

Este proyecto lo desarrollé como side project mientras trabajo 8 horas diarias en YPF. Lo construí en aproximadamente 10 días dedicándole entre 1.5 y 2 horas por noche. El contacto con el club llegó a través de un amigo que administra las canchas.

El tiempo acotado de desarrollo fue posible gracias a la elección del stack: Supabase eliminó la necesidad de armar un backend custom, Next.js App Router simplificó el routing y el data fetching, y Tailwind me permitió iterar sobre el diseño sin salir del markup.

Próximos pasos

- **Rollout completo** al total de socios del club.
- **Dashboard de métricas** para el administrador: reservas/mes, ocupación por cancha, horarios pico, tasa de cancelación.
- **Notificaciones push / recordatorios** previos al turno.
- **Expansión a otras canchas** del club (actualmente cubre tenis; el club también tiene otras disciplinas).

"Si te interesa ver el sistema en funcionamiento o tenés preguntas sobre la implementación, escribime."